

Document Number: P3323R0.

Date: 2024-06-10.

Reply to: Gonzalo Brito Gadeschi <gonzalob_at_nvidia.com (<http://nvidia.com>)>.

Authors: Gonzalo Brito Gadeschi, Lewis Baker.

Audience: SG1.

cv-qualified types in atomic and atomic_ref

- cv-qualified types in atomic and atomic_ref
 - Summary
 - Motivation
 - Resolution for std::atomic
 - Resolution for std::atomic_ref

Summary

Addresses LWG#4069 (<https://cplusplus.github.io/LWG/issue4069>) and LWG#3508 (<https://cplusplus.github.io/LWG/issue3508>) by clarifying that cv-qualified types are not supported by `std::atomic<T>` and specifying how these are supported by `std::atomic_ref<T>`.

Motivation

CWG#2094 (<https://cplusplus.github.io/CWG/issues/2094.html>) made `is_trivially_copyable_v<volatile ...-type>` (integer, pointer, floating-point) true, leading to LWG#3508 (<https://cplusplus.github.io/LWG/issue3508>) and LWG#4069 (<https://cplusplus.github.io/LWG/issue4069>).

Supporting `atomic_ref<volatile T>` can be useful for atomically accessing objects of type `T` stored in shared-memory where the object was not created as an `atomic<T>`.

Resolution for `std::atomic`

`std::atomic<...-type>` specializations only apply for cv-unqualified types.

- *Proposed resolution:* restrict `std::atomic<T>` to types `T` for which `same_as<T, remove_cv_t<T>>` is true.

- *Rationale:* `atomic<volatile int>` use case is served by `volatile atomic<int>`, i.e., there is no need to support `atomic<volatile T>`.
- *Impact:* `libstdc++` and `libc++` can't compile `atomic<volatile T>` already. MSVC can, but usage is limited, e.g., because `fetch_add` only exists on specialization, not primary template.
- *Proposed wording:*

Modify [atomics.types.generic.general] (<https://eel.is/c++draft/atomics.types.generic.general>):

The template argument for `T` shall meet the *Cpp17CopyConstructible* and *Cpp17CopyAssignable* requirements. The program is ill-formed if any of

1. `is_trivially_copyable_v<T>`,
 2. `is_copy_constructible_v<T>`,
 3. `is_move_constructible_v<T>`,
 4. `is_copy_assignable_v<T>`, ~~or~~
 5. `is_move_assignable_v<T>`, or
 6. `same_as<T, remove_cv_t<T>>`
- is false.

Resolution for `std::atomic_ref`

LWG#3508 (<https://cplusplus.github.io/LWG/issue3508>) also points out this problem, and indicates that for const-qualified types, it is not possible to implement atomic load or atomic read-modify-write operations.

`std::atomic_ref<...-type>` specializations only apply for cv-unqualified types.

- *Proposed resolution:* specify `std::atomic_ref<T>` for cv-qualified `T` by restricting support of `volatile`-qualified types to lock-free atomics and restricting support of `const`-qualified types to atomic read operations.
- *Rationale:* `atomic_ref` goal of improving concurrency support when interfacing with third-party types, which may be using `volatile int` for historical purposes, needs `std::atomic_ref<volatile int>`: the `atomic_ref` itself is not `volatile`, the data it references is.
- *Impact:* `libstdc++` and `libc++` (among others) would need to implement it.
- *Wording:*

Modify [atomics.ref.generic.general] (<https://eel.is/c++draft/atomics.ref.generic.general>):

```

namespace std {
    template<class T> struct atomic_ref {
    private:
        T* ptr;                // exposition only

    public:
        using value_type = Tremove_cv_t<T>;
        static constexpr size_t required_alignment = implementation-defined;

        static constexpr bool is_always_lock_free = implementation-defined;
        bool is_lock_free() const noexcept;

        explicit atomic_ref(T&);
        atomic_ref(const atomic_ref&) noexcept;
        atomic_ref& operator=(const atomic_ref&) = delete;

        void store(Tvalue_type, memory_order = memory_order::seq_cst) const noexcept;
        Tvalue_type operator=(Tvalue_type) const noexcept;
        Tvalue_type load(memory_order = memory_order::seq_cst) const noexcept;
        operator Tvalue_type() const noexcept;

        Tvalue_type exchange(Tvalue_type, memory_order = memory_order::seq_cst)
            const noexcept;
        bool compare_exchange_weak(Tvalue_type&, Tvalue_type,
                                   memory_order, memory_order)
            const noexcept;
        bool compare_exchange_strong(Tvalue_type&, Tvalue_type,
                                     memory_order, memory_order)
            const noexcept;
        bool compare_exchange_weak(Tvalue_type&, Tvalue_type,
                                   memory_order = memory_order::seq_cst)
            const noexcept;
        bool compare_exchange_strong(Tvalue_type&, Tvalue_type,
                                     memory_order = memory_order::seq_cst)
            const noexcept;

        void wait(Tvalue_type, memory_order = memory_order::seq_cst) const noexcept;
        void notify_one() const noexcept;
        void notify_all() const noexcept;
    };
}

```

1. An `atomic_ref` object applies atomic operations ([atomics.general]) to the object referenced by `*ptr` such that, for the lifetime ([basic.life]) of the `atomic_ref` object, the object referenced by `*ptr` is an atomic object ([intro.races]).

2. The program is ill-formed if `is_trivially_copyable_v<T>` is false.
3. The lifetime ([basic.life]) of an object referenced by `*ptr` shall exceed the lifetime of all `atomic_ref`s that reference the object. While any `atomic_ref` instances exist that reference the `*ptr` object, all accesses to that object shall exclusively occur through those `atomic_ref` instances. No subobject of the object referenced by `atomic_ref` shall be concurrently referenced by any other `atomic_ref` object.
4. Atomic operations applied to an object through a referencing `atomic_ref` are atomic with respect to atomic operations applied through any other `atomic_ref` referencing the same object.
[Note 1: Atomic operations or the `atomic_ref` constructor can acquire a shared resource, such as a lock associated with the referenced object, to enable atomic operations to be applied to the referenced object. — end note]
5. The program is ill-formed if `is_always_lock_free` is false and `is_volatile v<T>` is true.

Modify [atomics.ref.ops] (<https://eel.is/c++draft/atomics.ref.ops>) as follows:

33.5.7.2 Operations [atomics.ref.ops]

```
static constexpr size_t required_alignment;
```

1. The alignment required for an object to be referenced by an atomic reference, which is at least `alignof(T)`.
2. [Note 1: Hardware could require an object referenced by an `atomic_ref` to have stricter alignment ([basic.align]) than other objects of type `T`. Further, whether operations on an `atomic_ref` are lock-free could depend on the alignment of the referenced object. For example, lock-free operations on `std::complex<double>` could be supported only if aligned to `2*alignof(double)`. — end note]

```
static constexpr bool is_always_lock_free;
```

3. The static data member `is_always_lock_free` is `true` if the `atomic_ref` type's operations are always lock-free, and `false` otherwise.

```
bool is_lock_free() const noexcept;
```

4. Returns: `true` if operations on all objects of the type `atomic_ref<T>` are lock-free, `false` otherwise.

```
atomic_ref(T& obj);
```

- 5. Preconditions: The referenced object is aligned to `required_alignment`.
- 6. Postconditions: `*this` references `obj`.
- 7. Throws: Nothing.

```
atomic_ref(const atomic_ref& ref) noexcept;
```

- 8. Postconditions: `*this` references the object referenced by `ref`.

```
void store(constexpr value_type desired, memory_order order = memory_order::seq_cst) const
```

- 9. Constraints: `is_const v<T>` is false.

- 10. Preconditions: `order` is `memory_order::relaxed`, `memory_order::release`, or `memory_order::seq_cst`.
- 11. Effects: Atomically replaces the value referenced by `*ptr` with the value of `desired`. Memory is affected according to the value of `order`.

```
constexpr value_type operator=(constexpr value_type desired) const noexcept;
```

- 12. Constraints: `is_const v<T>` is false.

- 13. Effects: Equivalent to:

```
store(desired);  
return desired;
```

```
constexpr value_type load(memory_order order = memory_order::seq_cst) const noexcept;
```

- 14. Preconditions: `order` is `memory_order::relaxed`, `memory_order::consume`, `memory_order::acquire`, or `memory_order::seq_cst`.
- 15. Effects: Memory is affected according to the value of `order`.
- 16. Returns: Atomically returns the value referenced by `*ptr`.

```
operator constexpr value_type() const noexcept;
```

15. Effects: Equivalent to: `return load();`

```
Tvalue_type exchange(Tvalue_type desired, memory_order order = memory_order::seq_c
```

16. Constraints: `is_const_v<T>` is false.

17. Effects: Atomically replaces the value referenced by `*ptr` with `desired`. Memory is affected according to the value of `order`. This operation is an atomic read-modify-write operation ([intro.multithread]).

18. Returns: Atomically returns the value referenced by `*ptr` immediately before the effects.

```
bool compare_exchange_weak(Tvalue_type& expected, Tvalue_type desired,  
                           memory_order success, memory_order failure) const noexcept
```

```
bool compare_exchange_strong(Tvalue_type& expected, Tvalue_type desired,  
                             memory_order success, memory_order failure) const noexcept
```

```
bool compare_exchange_weak(Tvalue_type& expected, Tvalue_type desired,  
                           memory_order order = memory_order::seq_cst) const noexcept
```

```
bool compare_exchange_strong(Tvalue_type& expected, Tvalue_type desired,  
                             memory_order order = memory_order::seq_cst) const noexcept
```

19. Constraints: `is_const_v<T>` is false.

20. Preconditions: `failure` is `memory_order::relaxed`, `memory_order::consume`, `memory_order::acquire`, or `memory_order::seq_cst`.

21. Effects: Retrieves the value in `expected`. It then atomically compares the value representation of the value referenced by `*ptr` for equality with that previously retrieved from `expected`, and if `true`, replaces the value referenced by `*ptr` with that in `desired`. If and only if the comparison is `true`, memory is affected according to the value of `success`, and if the comparison is `false`, memory is affected according to the value of `failure`. When only one `memory_order` argument is supplied, the value of `success` is `order`, and the value of `failure` is `order` except that a value of `memory_order::acq_rel` shall be replaced by the value `memory_order::acquire` and a value of `memory_order::release` shall be replaced by the value `memory_order::relaxed`. If and only if the comparison is `false` then, after the atomic operation, the value in `expected` is replaced by the value read from the value referenced by `*ptr` during the atomic comparison. If the operation returns `true`, these operations are

atomic read-modify-write operations ([intro.races]) on the value referenced by `*ptr` . Otherwise, these operations are atomic load operations on that memory.

22. Returns: The result of the comparison.

23. Remarks: A weak compare-and-exchange operation may fail spuriously. That is, even when the contents of memory referred to by `expected` and `ptr` are equal, it may return `false` and store back to `expected` the same memory contents that were originally there. [Note 2: This spurious failure enables implementation of compare-and-exchange on a broader class of machines, e.g., load-locked store-conditional machines. A consequence of spurious failure is that nearly all uses of weak compare-and-exchange will be in a loop. When a compare-and-exchange is in a loop, the weak version will yield better performance on some platforms. When a weak compare-and-exchange would require a loop and a strong one would not, the strong one is preferable. — end note]

```
void wait(¶value_type old, memory_order order = memory_order::seq_cst) const noexcept
```

22. Preconditions: `order` is `memory_order::relaxed` , `memory_order::consume` , `memory_order::acquire` , or `memory_order::seq_cst` .

23. Effects: Repeatedly performs the following steps, in order:

(23.1) Evaluates `load(order)` and compares its value representation for equality against that of `old` .

(23.2) If they compare unequal, returns.

(23.3) Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.

24. Remarks: This function is an atomic waiting operation ([atomics.wait]) on atomic object `*ptr` .

```
void notify_one() const noexcept;
```

25. Effects: Unblocks the execution of at least one atomic waiting operation on `*ptr` that is eligible to be unblocked ([atomics.wait]) by this call, if any such atomic waiting operations exist.

26. Remarks: This function is an atomic notifying operation ([atomics.wait]) on atomic object `*ptr` .

```
void notify_all() const noexcept;
```

27. Effects: Unblocks the execution of all atomic waiting operations on `*ptr` that are eligible to be unblocked (`[atomics.wait]`) by this call.
28. Remarks: This function is an atomic notifying operation (`[atomics.wait]`) on atomic object `*ptr`.

Modify `[atomics.ref.int]` (<https://eel.is/c++draft/atomics.ref.int>):

33.5.7.3 Specializations for integral types`[atomics.ref.int]`

1. There are specializations of the `atomic_ref` class template for all integral types except `cv bool` ~~the integral types `char`, `signed char`, `unsigned char`, `short`, `unsigned short`, `int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, `char8_t`, `char16_t`, `char32_t`, `wchar_t`, and any other types needed by the typedefs in the header `<stdint>`~~. For each such possibly cv-qualified type *integral-type*, the specialization `atomic_ref<integral-type>` provides additional atomic operations appropriate to integral types.

[Note 1: The specialization `atomic_ref<bool>` uses the primary template (`[atomics.ref.generic]`). — end note]

2. The program is ill-formed if `is always lock free` is `false` and `is volatile v<T>` is `true`.

```

namespace std {
    template<> struct atomic_ref<integral-type> {
    private:
        integral-type* ptr;          // exposition only

    public:
        using value_type = remove_cv_t<integral-type>;
        using difference_type = value_type;
        static constexpr size_t required_alignment = implementation-defined;

        static constexpr bool is_always_lock_free = implementation-defined;
        bool is_lock_free() const noexcept;

        explicit atomic_ref(integral-type&);
        atomic_ref(const atomic_ref&) noexcept;
        atomic_ref& operator=(const atomic_ref&) = delete;

        void store(integral-typevalue type, memory_order = memory_order::seq_cst) const
        integral-typevalue type operator=(integral-typevalue type) const noexcept;
        integral-typevalue type load(memory_order = memory_order::seq_cst) const noexcept;
        operator integral-typevalue type() const noexcept;

        integral-typevalue type exchange(integral-typevalue type,
                                         memory_order = memory_order::seq_cst) const noexcept;
        bool compare_exchange_weak(integral-typevalue type&, integral-typevalue type,
                                   memory_order, memory_order) const noexcept;
        bool compare_exchange_strong(integral-typevalue type&, integral-typevalue type,
                                     memory_order, memory_order) const noexcept;
        bool compare_exchange_weak(integral-typevalue type&, integral-typevalue type,
                                   memory_order = memory_order::seq_cst) const noexcept;
        bool compare_exchange_strong(integral-typevalue type&, integral-typevalue type,
                                     memory_order = memory_order::seq_cst) const noexcept;

        integral-typevalue type fetch_add(integral-typevalue type,
                                         memory_order = memory_order::seq_cst) const noexcept;
        integral-typevalue type fetch_sub(integral-typevalue type,
                                         memory_order = memory_order::seq_cst) const noexcept;
        integral-typevalue type fetch_and(integral-typevalue type,
                                         memory_order = memory_order::seq_cst) const noexcept;
        integral-typevalue type fetch_or(integral-typevalue type,
                                         memory_order = memory_order::seq_cst) const noexcept;
        integral-typevalue type fetch_xor(integral-typevalue type,
                                         memory_order = memory_order::seq_cst) const noexcept;
        integral-typevalue type fetch_max(integral-typevalue type,
                                         memory_order = memory_order::seq_cst) const noexcept;
        integral-typevalue type fetch_min(integral-typevalue type,
                                         memory_order = memory_order::seq_cst) const noexcept;

        integral-typevalue type operator++(int) const noexcept;

```

```

integral_typevalue_type operator--(int) const noexcept;
integral_typevalue_type operator++() const noexcept;
integral_typevalue_type operator--() const noexcept;
integral_typevalue_type operator+=(integral_typevalue_type) const noexcept;
integral_typevalue_type operator-=(integral_typevalue_type) const noexcept;
integral_typevalue_type operator&=(integral_typevalue_type) const noexcept;
integral_typevalue_type operator|=(integral_typevalue_type) const noexcept;
integral_typevalue_type operator^=(integral_typevalue_type) const noexcept;

void wait(integral_typevalue_type, memory_order = memory_order::seq_cst) const
void notify_one() const noexcept;
void notify_all() const noexcept;
};
}

```

3. Descriptions are provided below only for members that differ from the primary template.
4. The following operations perform arithmetic computations. The correspondence among key, operator, and computation is specified in Table 148.

```

integral_typevalue_type fetch_key(integral_typevalue_type operand,
memory_order order = memory_order::seq_cst) const noexcept;

```

5. Constraints: `is_const_v<integral_type>` is false .
6. Effects: Atomically replaces the value referenced by `*ptr` with the result of the computation applied to the value referenced by `*ptr` and the given `operand` . Memory is affected according to the value of `order` . These operations are atomic read-modify-write operations ([intro.races]).
7. Returns: Atomically, the value referenced by `*ptr` immediately before the effects.
8. Remarks: Except for `fetch_max` and `fetch_min` , for signed integer types the result is as if the object value and parameters were converted to their corresponding unsigned types, the computation performed on those types, and the result converted back to the signed type.
[Note 2: There are no undefined results arising from the computation. — end note]
9. For `fetch_max` and `fetch_min` , the maximum and minimum computation is performed as if by `max` and `min` algorithms ([alg.min.max]), respectively, with the object value and the first parameter as the arguments.

```

integral_typevalue_type operator op=(integral_typevalue_type operand) const noe

```

10. Constraints: `is_const v<integral-type>` is `false` .

11. Effects: Equivalent to: `return fetch_key(operand) op operand;`

Modify [atomics.ref.float] (<https://eel.is/c++draft/atomics.ref.float>):

33.5.7.4 Specializations for floating-point types[atomics.ref.float]

1. There are specializations of the `atomic_ref` class template for all ~~cv-qualified~~ floating-point types. For each such possibly cv-qualified type *floating-point-type*, the specialization `atomic_ref<floating-point>` provides additional atomic operations appropriate to floating-point types.
2. The program is ill-formed if `is_always_lock_free` is `false` and `is_volatile v<T>` is `true` .

```

namespace std {
    template<> struct atomic_ref<floating-point-type> {
    private:
        floating-point-type* ptr;    // exposition only

    public:
        using value_type = remove_cv_t<floating-point-type>;
        using difference_type = value_type;
        static constexpr size_t required_alignment = implementation-defined;

        static constexpr bool is_always_lock_free = implementation-defined;
        bool is_lock_free() const noexcept;

        explicit atomic_ref(floating-point-type&);
        atomic_ref(const atomic_ref&) noexcept;
        atomic_ref& operator=(const atomic_ref&) = delete;

        void store(floating-point-typevalue_type, memory_order = memory_order::seq_cst)
        floating-point-typevalue_type operator=(floating-point-typevalue_type) const
        floating-point-typevalue_type load(memory_order = memory_order::seq_cst) const
        operator floating-point-typevalue_type() const noexcept;

        floating-point-typevalue_type exchange(floating-point-typevalue_type,
            memory_order = memory_order::seq_cst) const noexcept;
        bool compare_exchange_weak(floating-point-typevalue_type&, floating-point-typevalue_type,
            memory_order, memory_order) const noexcept;
        bool compare_exchange_strong(floating-point-typevalue_type&, floating-point-typevalue_type,
            memory_order, memory_order) const noexcept;
        bool compare_exchange_weak(floating-point-typevalue_type&, floating-point-typevalue_type,
            memory_order = memory_order::seq_cst) const noexcept;
        bool compare_exchange_strong(floating-point-typevalue_type&, floating-point-typevalue_type,
            memory_order = memory_order::seq_cst) const noexcept;

        floating-point-typevalue_type fetch_add(floating-point-typevalue_type,
            memory_order = memory_order::seq_cst) const noexcept;
        floating-point-typevalue_type fetch_sub(floating-point-typevalue_type,
            memory_order = memory_order::seq_cst) const noexcept;

        floating-point-typevalue_type operator+=(floating-point-typevalue_type) const
        floating-point-typevalue_type operator-=(floating-point-typevalue_type) const

        void wait(floating-point-typevalue_type, memory_order = memory_order::seq_cst)
        void notify_one() const noexcept;
        void notify_all() const noexcept;
    };
}

```

3. Descriptions are provided below only for members that differ from the primary template.
4. The following operations perform arithmetic computations. The correspondence among key, operator, and computation is specified in Table 148.

```
floating-point-typevalue_type fetch_key(floating-point-typevalue_type operand,
                                     memory_order order = memory_order::seq_cst) const noexcept
```

5. Constraints: `is_const_v<floating-point-type>` is false.
6. Effects: Atomically replaces the value referenced by `*ptr` with the result of the computation applied to the value referenced by `*ptr` and the given `operand`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations ([intro.races]).
7. Returns: Atomically, the value referenced by `*ptr` immediately before the effects.
8. Remarks: If the result is not a representable value for its type ([expr.pre]), the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on *floating-point-type* should conform to the `std::numeric_limits<floating-point-typevalue_type>` traits associated with the floating-point type ([limits.syn]). The floating-point environment ([cfenv]) for atomic arithmetic operations on *floating-point-type* may be different than the calling thread's floating-point environment.

```
floating-point-typevalue_type operator op=(floating-point-typevalue_type operand)
```

9. Constraints: `is_const_v<floating-point-type>` is false.
 10. Effects: Equivalent to: `return fetch_key(operand) op operand;`
- Modify [atomics.ref.pointer] (<https://eel.is/c++draft/atomics.ref.pointer>):

33.5.7.5 Partial specialization for pointers[atomics.ref.pointer]

1. There are specializations of the `atomic_ref` class template for all pointer-to-object types. For each such possibly cv-qualified type *pointer-type*, the specialization `atomic_ref<pointer-type>` provides additional atomic operations appropriate to pointer types.
2. The program is ill-formed if `is_always_lock_free` is false and `is_volatile_v<T>` is true.

```

namespace std {
    template<class T> struct atomic_ref<T*pointer-type> {
    private:
        T*pointer-type* ptr;          // exposition only

    public:
        using value_type = T*remove_cv_t<pointer-type>;
        using difference_type = ptrdiff_t;
        static constexpr size_t required_alignment = implementation-defined;

        static constexpr bool is_always_lock_free = implementation-defined;
        bool is_lock_free() const noexcept;

        explicit atomic_ref(T*pointer-type&);
        atomic_ref(const atomic_ref&) noexcept;
        atomic_ref& operator=(const atomic_ref&) = delete;

        void store(T*value_type, memory_order = memory_order::seq_cst) const noexcept;
        T*value_type operator=(T*value_type) const noexcept;
        T*value_type load(memory_order = memory_order::seq_cst) const noexcept;
        operator T*value_type() const noexcept;

        T*value_type exchange(T*value_type, memory_order = memory_order::seq_cst) const;
        bool compare_exchange_weak(T*value_type&, T*value_type,
                                   memory_order, memory_order) const noexcept;
        bool compare_exchange_strong(T*value_type&, T*value_type,
                                     memory_order, memory_order) const noexcept;
        bool compare_exchange_weak(T*value_type&, T*value_type,
                                   memory_order = memory_order::seq_cst) const noexcept;
        bool compare_exchange_strong(T*value_type&, T*value_type,
                                     memory_order = memory_order::seq_cst) const noexcept;

        T*value_type fetch_add(difference_type, memory_order = memory_order::seq_cst)
        T*value_type fetch_sub(difference_type, memory_order = memory_order::seq_cst)
        T*value_type fetch_max(T*value_type, memory_order = memory_order::seq_cst) const
        T*value_type fetch_min(T*value_type, memory_order = memory_order::seq_cst) const

        T*value_type operator++(int) const noexcept;
        T*value_type operator--(int) const noexcept;
        T*value_type operator++() const noexcept;
        T*value_type operator--() const noexcept;
        T*value_type operator+=(difference_type) const noexcept;
        T*value_type operator-=(difference_type) const noexcept;

        void wait(T*value_type, memory_order = memory_order::seq_cst) const noexcept;
        void notify_one() const noexcept;
        void notify_all() const noexcept;
    };
}

```

```
};
}
```

3. Descriptions are provided below only for members that differ from the primary template.
4. The following operations perform arithmetic computations. The correspondence among key, operator, and computation is specified in Table 149.

$\mp$ value_type fetch_key(difference_type operand, memory_order order = memory_order:

5. Constraints: *is_const_v<pointer-type>* is false.
6. Mandates: $\mp$ remove_pointer t<pointer-type> is a complete object type.
7. Effects: Atomically replaces the value referenced by *ptr with the result of the computation applied to the value referenced by *ptr and the given operand. Memory is affected according to the value of order. These operations are atomic read-modify-write operations ([intro.races]).
8. Returns: Atomically, the value referenced by *ptr immediately before the effects.
9. Remarks: The result may be an undefined address, but the operations otherwise have no undefined behavior.
10. For fetch_max and fetch_min, the maximum and minimum computation is performed as if by max and min algorithms ([alg.min.max]), respectively, with the object value and the first parameter as the arguments.
[Note 1: If the pointers point to different complete objects (or subobjects thereof), the < operator does not establish a strict weak ordering (Table 29, [expr.rel]). — end note]

$\mp$ value_type operator op=(difference_type operand) const noexcept;

11. Constraints: *is_const_v<pointer-type>* is false.
12. Effects: Equivalent to: return fetch_key(operand) op operand;

Modify [atomics.ref.memop] (<https://eel.is/c++draft/atomics.ref.memop>):

33.5.7.6 Member operators common to integers and pointers to objects[atomics.ref.memop]

1. Let referred-type be pointer-type for the specializations in [atomics.ref.pointer] and be integral-type for the specializations in [atomics.ref.int].

`value_type operator++(int) const noexcept;`

2. Constraints: `is_const v<referred-type>` is false.

3. Effects: Equivalent to: `return fetch_add(1);`

`value_type operator--(int) const noexcept;`

4. Constraints: `is_const v<referred-type>` is false.

5. Effects: Equivalent to: `return fetch_sub(1);`

`value_type operator++() const noexcept;`

6. Constraints: `is_const v<referred-type>` is false.

7. Effects: Equivalent to: `return fetch_add(1) + 1;`

`value_type operator--() const noexcept;`

8. Constraints: `is_const v<referred-type>` is false.

9. Effects: Equivalent to: `return fetch_sub(1) - 1;`